



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده‌ی ریاضی و علوم کامپیوتر

پروژه کارشناسی

بررسی و ارزیابی روش PINNsFormer در حل معادلات دیفرانسیل با مشتقات جزئی

استاد راهنما
دکتر فاطمه شاکری

نگارش
ماریا مهدوی لاین
زهرة سرملى سعیدی

مرداد ۱۴۰۵

چکیده

شبکه‌های عصبی مبتنی بر فیزیک^۱ در سال‌های اخیر، به یکی از روش‌های پرکاربرد برای حل معادلات دیفرانسیل با مشتقات جزئی تبدیل شده‌اند. با توجه به این نکته، عملکرد PINN‌های متداول در برخی مسائل، به‌ویژه معادلات دیفرانسیل سخت که دارای مؤلفه‌های با فرکانس بالا، می‌تواند افت کند. یکی از دلایلی که برای این موضوع مطرح شده، نحوه‌ی پردازش ورودی‌ها در معماری استاندارد PINN است. در این معماری، ارتباط میان نقاط زمانی مختلف به‌طور مستقیم در ساختار شبکه در نظر گرفته نمی‌شود. مدل PINNsFormer با استفاده از یک دنباله‌ی شبه‌زمانی و مکانیزم خودتوجهی، تلاش می‌کند این وابستگی را در فرایند یادگیری وارد کند. در این پروژه، این مدل روی پنج معادله‌ی دیفرانسیل جزئی با ویژگی‌های متفاوت، شامل برگرز، آلن-کان، شرودینگر غیرخطی، کوراموتو-سیواشینسکی و کورتوگ-دووریس، پیاده‌سازی و با یک PINN استاندارد مقایسه شده است. معیار اصلی مقایسه، خطای نسبی پاسخ عددی است و در کنار آن، هزینه‌ی محاسباتی نیز بررسی شده‌اند. در آزمایش‌های انجام‌شده، PINNsFormer در هر پنج مسئله خطای کمتری از مدل مبنا به دست آورد و کاهش خطای نسبی آن، بسته به معادله، از حدود 54% تا بیش از 99% متغیر بود. بیشترین بهبود در آزمایش معادله‌ی کوراموتو-سیواشینسکی مشاهده شد؛ در این آزمایش، خطای نسبی از حدود 0.30 برای PINN به 0.0001 برای PINNsFormer کاهش یافت.

کلیدواژه‌ها: شبکه‌های عصبی مبتنی بر فیزیک، معادلات دیفرانسیل با مشتقات جزئی، معماری ترنسفورمر، مکانیزم توجه

¹Physics-Informed Neural Networks (PINNs)

۱ مقدمه

۱.۱ بیان مسئله

معادلات دیفرانسیل با مشتقات جزئی^۲ ابزار ریاضی مهمی برای توصیف بسیاری از پدیده‌های علوم و مهندسی هستند؛ برای نمونه، انتقال حرارت، جریان سیالات، مکانیک کوانتومی و برخی مدل‌های زیستی را می‌توان با این معادلات بیان کرد. برای تعداد محدودی از مسائل ساده جواب تحلیلی در دسترس است، اما در بسیاری از مسائل واقعی باید از روش‌های عددی استفاده کرد. روش‌هایی مانند تفاضل محدود^۳، اجزای محدود^۴ از روش‌های شناخته‌شده برای این کار هستند. با این حال، در مسائلی با ابعاد بالا یا هندسه‌های پیچیده، هزینه ساخت و نگهداری مش و هزینه محاسبات افزایش پیدا می‌کند.

هم‌زمان با پیشرفت یادگیری ماشین علمی^۵، استفاده از شبکه‌های عصبی برای تقریب جواب معادلات دیفرانسیل نیز مورد توجه قرار گرفته است. در این روش‌ها، به جای آن که دامنه‌ی مسئله ابتدا به یک مش گسسته تبدیل شود، جواب با یک شبکه‌ی عصبی نمایش داده می‌شود. مشتق‌های مورد نیاز برای تشکیل معادله نیز می‌تواند با مشتق‌گیری خودکار محاسبه شوند. این ویژگی، استفاده از شبکه‌های عصبی را برای برخی مسائل با ابعاد بالا مناسب می‌کند، هرچند خود این روش‌ها نیز محدودیت‌ها و چالش‌های عددی خاصی دارند.

۲.۱ محدودیت‌های شناخته‌شده‌ی PINN

ایده‌ی اصلی این روش آن است که شبکه تنها با داده‌های شرط اولیه و مرزی آموزش نبیند، بلکه باقی‌مانده‌ی^۶ معادله‌ی دیفرانسیل نیز در تابع هزینه وارد شود. در نتیجه، فرایند آموزش شبکه به گونه‌ای انجام می‌شود که معادله و شرایط مسئله را برآورده کند.

با وجود این، پژوهش‌های بعدی نشان داده‌اند که PINN‌های مبتنی بر پرسپترون چندلایه‌ی استاندارد^۸ در بعضی مسائل عملکرد مطلوبی ندارند. این مشکل در معادلات سخت^۹، مؤلفه‌های با فرکانس بالا و رفتار چندمقیاسی بیشتر دیده می‌شود [۸]. در چنین شرایطی، فرایند بهینه‌سازی ممکن است به تقریب نامناسبی از جواب برسد؛ برای مثال، در مسائل دارای رفتارهای نوسانی یا تغییرات سریع، شبکه ممکن است در بازنمایی مؤلفه‌های با فرکانس بالا دچار مشکل شود و در نتیجه، به سمت تقریب‌های بیش از حد هموار میل کند. از سوی دیگر، اختلاف مقیاس گرادینت‌های مربوط به مؤلفه‌های مختلف تابع هزینه می‌تواند موجب نامتوازن شدن فرایند بهینه‌سازی شود؛ به گونه‌ای که برخی مؤلفه‌ها بر روند آموزش غالب شده و یادگیری سایر مؤلفه‌ها با کندی بیشتری انجام شود.

نکته‌ی دیگری که در اینجا اهمیت دارد، نحوه‌ی نمایش وابستگی زمانی در معماری استاندارد است. در PINN معمولی، هر زوج (x, t) مستقیماً به شبکه داده می‌شود و خروجی $\hat{u}(x, t)$ محاسبه می‌شود. در نتیجه، رابطه‌ی یک نقطه با نقاط زمانی اطراف آن در ساختار شبکه به صورت صریح نمایش داده نمی‌شود. این موضوع به‌ویژه در مسائلی که رفتار جواب در طول زمان اهمیت زیادی دارد، می‌تواند یک محدودیت باشد. PINNsFormer با تغییر همین بخش از معماری، یعنی تبدیل هر ورودی به یک دنباله‌ی کوتاه و استفاده از مکانیزم توجه، تلاش می‌کند اطلاعات مربوط به زمان‌های نزدیک را در فرایند یادگیری وارد کند.

^۲Partial Differential Equations (PDEs)^۳Finite Difference^۴Finite Element^۵Spectral Methods^۶Scientific Machine Learning^۷Residual^۸Multi-Layer Perceptron (MLP)^۹Stiff

۳.۱ اهداف پروژه

هدف این پروژه، پیاده‌سازی و ارزیابی چارچوب PINNsFormer و مقایسه‌ی عملکرد آن با PINN استاندارد در حل معادلات دیفرانسیل جزئی است. در این راستا، ابتدا روش بر اساس مقاله‌ی اصلی بازتولید شده و سپس عملکرد آن روی پنج معادله‌ی متفاوت، شامل برگرز، آلن-کان، شرودینگر غیرخطی، کوراموتو-سیواشینسکی و کورتوگ-دوورس، بررسی می‌شود. همچنین، میزان بهبود خطا و ارتباط آن با ویژگی‌های مسئله و دشواری آموزش PINN مورد ارزیابی قرار می‌گیرد.

۲ مبانی نظری و مطالعات پیشین

۱.۲ یادگیری عمیق برای حل معادلات دیفرانسیل با مشتقات جزئی

در دهه‌ی گذشته، دو مسیر اصلی برای استفاده از یادگیری عمیق در این حوزه وجود دارد، نخست یادگیری عملگر^{۱۰} است که به‌جای تقریب یک جواب مشخص، نگاشت بین فضاها را یاد می‌گیرد؛ برای مثال، نگاشتی از شرط اولیه یا پارامترهای مسئله به تابع جواب معادله. از روش‌های مهم این رویکرد می‌توان به DeepONet [۴] و Fourier Neural Operator [۵] اشاره کرد. DeepONet این نگاشت را با استفاده از دو شبکه‌ی عصبی برای نمایش ورودی تابعی و محل ارزیابی تقریب می‌زند، در حالی که FNO با یادگیری در فضای فوریه، امکان مدل‌سازی عملگرهای دیفرانسیلی را فراهم می‌کند. مسیر دوم، که مبنای این پروژه است، شبکه‌های عصبی مبتنی بر فیزیک نام دارد. این مسیر به‌جای یادگیری یک نگاشت کلی، مستقیماً جواب یک مسئله‌ی مشخص را تقریب می‌زند، آن هم با تحمیل قید فیزیکی معادله بر خروجی شبکه.

۲.۲ شبکه‌های عصبی مبتنی بر فیزیک: مبانی و سیر تحول

ایده‌ی استفاده از شبکه‌های عصبی برای حل معادلات دیفرانسیل به مقاله‌ی لاگاریس [۳] در دهه‌ی ۱۹۹۰ برمی‌گردد؛ آن‌ها شبکه‌ای ساده برای حل معادلات دیفرانسیل معمولی و جزئی با شرایط مرزی ساده به کار بردند. بعدها، با ظهور مشتق‌گیری خودکار^{۱۱} در مدل‌های یادگیری عمیق مدرن، [۲] این ایده را با عنوان شبکه‌های عصبی مبتنی بر فیزیک (PINN) به شکل امروزی‌اش بازتعریف کردند و این مدل به‌سرعت شناخته شد. در این مدل، جواب $u(\mathbf{x}, t)$ با یک شبکه‌ی عصبی $\hat{u}_\theta$ نمایش داده می‌شود و تابع هزینه ترکیبی از باقی‌مانده‌ی معادله (محاسبه‌شده با مشتق‌گیری خودکار روی نقاط نمونه‌برداری‌شده داخل دامنه) و خطای شرایط مرزی/اولیه است:

$$\mathcal{L}_{\text{PINN}}(\theta) = \lambda_r \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} \|\mathcal{D}[\hat{u}_\theta](\mathbf{x}_i, t_i) - f(\mathbf{x}_i, t_i)\|^2}_{\text{باقی‌مانده‌ی معادله}} + \lambda_b \underbrace{\frac{1}{N_b} \sum_{i=1}^{N_b} \|\mathcal{B}[\hat{u}_\theta](\mathbf{x}_i, t_i) - g(\mathbf{x}_i, t_i)\|^2}_{\text{شرط مرزی/اولیه}}. \quad (۱)$$

این فرمول‌بندی ساده، که نیازی به تولید مش یا داده‌ی برجسب‌دار ندارد، باعث شد این رویکرد به‌سرعت در حوزه‌های مختلف، از مکانیک سیالات تا زیست‌پزشکی، گسترش پیدا کند. کتابخانه‌هایی مانند DeepXDE [۶] نیز پیاده‌سازی آن را در دسترس همه‌ی پژوهشگران قرار دادند.

۳.۲ ریشه‌های ناکارآمدی PINN: از سوگیری طیفی تا آسیب‌شناسی گرادینان

مشاهده‌ی تکرارشونده این است که PINN استاندارد، در دسته‌ای خاص از مسائل، به‌جای رسیدن به جواب واقعی، به یک تقریب بیش‌ازحد صاف میل می‌کند. دلایل زیر ریشه این پدیده را توضیح می‌دهد:

¹⁰Operator Learning

¹¹Automatic Differentiation

سوگیری طیفی شبکه‌های عصبی. رهامن و همکاران [۷] نشان دادند که شبکه‌های عصبی، در تقریب یک تابع، مؤلفه‌های با فرکانس پایین را سریع‌تر از مؤلفه‌های با فرکانس بالا یاد می‌گیرند. این ویژگی باعث می‌شود تقریب بخش‌های پرنوسان و دارای تغییرات سریع برای شبکه دشوارتر باشد. از آنجا که جواب برخی PDEها، به‌ویژه در حضور همرفت قوی یا لایه‌های مرزی تیز، دارای مؤلفه‌های فرکانس بالا است، سوگیری طیفی می‌تواند دقت PINN را در تقریب این جواب‌ها کاهش داده و فرایند آموزش را دشوارتر کند.

دشواری بهینه‌سازی تابع هزینه. کریشناپریان و همکاران [۸] با بررسی معادلات همرفت^{۱۲} و واکنش-نفوذ^{۱۳} نشان دادند که ناکارآمدی PINN الزاماً ناشی از ظرفیت محدود شبکه نیست، بلکه می‌تواند از ساختار تابع هزینه‌ی ترکیبی ناشی شود. با افزایش پیچیدگی فیزیکی مسئله، تابع هزینه می‌تواند دارای ساختاری به‌شدت غیرمحدب و نواحی با هندسه‌ی نامطلوب باشد؛ در نتیجه، روش‌های بهینه‌سازی مبتنی بر گرادیان ممکن است در نقاط بحرانی نامطلوب گرفتار شوند و همگرایی به جواب دقیق با دشواری مواجه شود.

۴.۲ مدل‌سازی دنباله‌ای و مکانیزم توجه در مسائل وابسته به زمان

در مسائل وابسته به زمان، مقدار جواب در یک نقطه از فضا و زمان به مقادیر آن در نقاط زمانی دیگر وابسته است. معماری ترنسفورمر [۱۳] با استفاده از مکانیزم خودتوجهی^{۱۴} می‌تواند این وابستگی‌ها را مستقیماً بین عناصر یک دنباله مدل کند، بدون آن‌که مانند شبکه‌های بازگشتی، دنباله را به‌صورت گام‌به‌گام پردازش کند. با این حال، ورودی معمول PINN یک نقطه‌ی (x, t) است و به‌طور طبیعی ساختار دنباله‌ای ندارد.

در روش PINNsFormer [۱]، هر نقطه‌ی (x, t) با در نظر گرفتن چند زمان مجاور به یک دنباله‌ی کوتاه^{۱۵} تبدیل می‌شود. سپس معماری مبتنی بر Self-Attention روابط میان این نقاط زمانی را یاد می‌گیرد و اطلاعات زمانی را در تقریب جواب وارد می‌کند. در نتیجه، شبکه به‌جای پردازش مستقل هر نقطه‌ی فضازمانی، می‌تواند وابستگی میان چند زمان متوالی را در فرایند یادگیری لحاظ کند.

۳ معماری و مکانیزم‌های مدل PINNsFormer

۱.۳ فرمول‌بندی کلی مسئله

در این بخش، نمادها و فرمول‌بندی ریاضی‌ای که در ادامه‌ی این فصل به کار می‌رود معرفی می‌شود. معادلات دیفرانسیل با مشتقات جزئی این پروژه همگی از فرم کلی زیر پیروی می‌کنند:

$$\mathcal{D}[u(x, t)] = f(x, t), \quad \forall (x, t) \in \Omega \quad (۲)$$

$$\mathcal{B}[u(x, t)] = g(x, t), \quad \forall (x, t) \in \partial\Omega \quad (۳)$$

$$u(x, 0) = h(x), \quad \forall x \quad (۴)$$

که در آن $\Omega \subset \mathbb{R}^2$ دامنه‌ی مکانی-زمانی مسئله، $\partial\Omega$ مرز آن، $\mathcal{D}$ عملگر دیفرانسیلی (خطی یا غیرخطی) معادله، و $\mathcal{B}$ عملگر شرط مرزی است. هدف پیدا کردن تابعی مانند $\hat{u}_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}$ است که با یک شبکه‌ی عصبی با پارامترهای θ نمایش داده می‌شود، به شکلی که هر سه رابطه‌ی بالا را تقریباً برآورده کند. تفاوت اصلی PINN استاندارد و PINNsFormer، همان‌طور که در ادامه‌ی این فصل نشان داده می‌شود، در نحوه‌ی نمایش $\hat{u}_\theta$ و ساختار تابع هزینه است، نه در خود فرمول‌بندی مسئله.

برای نمونه‌برداری عددی از این فرمول‌بندی پیوسته، سه مجموعه نقطه تعریف می‌شود: نقاط باقی‌مانده

^{۱۲}Convection

^{۱۳}Reaction-Diffusion

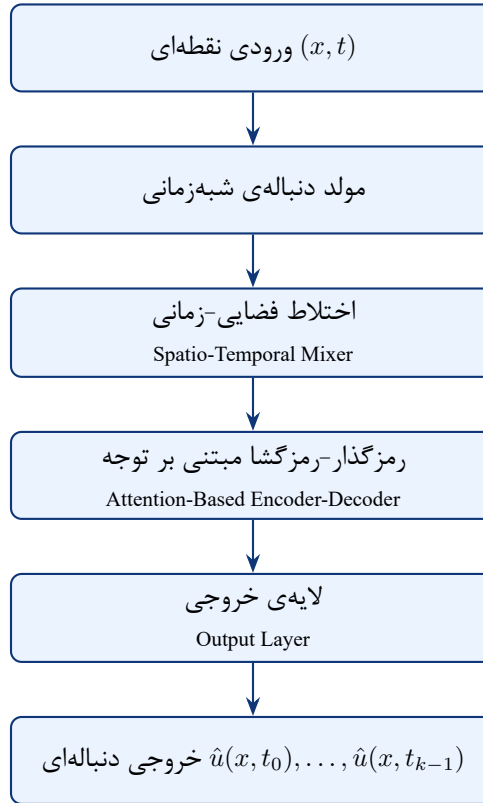
^{۱۴}Self-Attention

^{۱۵}Pseudo-Sequence

نقاط اولیه $\{x_i\}_{i=1}^{N_{ic}}$ (برای معادله‌ی (۴)). این نقاط، همان نقاط هم مکانی^{۱۶} هستند. و نقاط مرزی $\{(x_i, t_i)\}_{i=1}^{N_b} \subset \partial\Omega$ (برای معادله‌ی (۳))، و $\{(x_i, t_i)\}_{i=1}^{N_r} \subset \Omega$ (برای برآورده کردن معادله‌ی (۲))،

۲.۳ نمای کلی معماری

شکل ۱ مسیری را که یک ورودی نقطه‌ای از ابتدا تا انتهای شبکه طی می‌کند نشان می‌دهد: یک نقطه‌ی ورودی (x, t) به ترتیب از چهار جزء اصلی عبور می‌کند: مولد دنباله، لایه‌ی اختلاط، رمزگذار-رمزگشا، و لایه‌ی خروجی، تا در نهایت به تقریب جواب معادله تبدیل شود.



شکل ۱: نمایش مراحل کلی معماری

۳.۳ گام اول: مولد دنباله‌ی شبه‌زمانی

مشکل PINN استاندارد این است که هر نقطه‌ی (x, t) را مستقل از سایر نقاط زمانی پردازش می‌کند، در حالی که مکانیزم توجه به یک دنباله نیاز دارد. از آنجا که ورودی طبیعی PDE یک نقطه است، مولد دنباله‌ی شبه‌زمانی^{۱۷} با گسترش نقطه‌ی ورودی در چند گام زمانی، یک دنباله‌ی مصنوعی ایجاد می‌کند تا امکان استفاده از مکانیزم توجه فراهم شود.

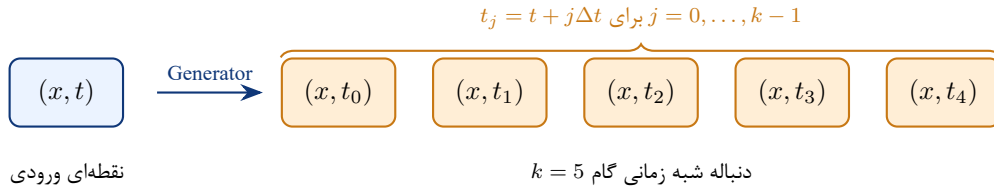
فرمول‌بندی. برای یک ورودی مکانی x و زمانی t ، و با دو ابرپارامتر k (طول دنباله) و Δt (گام زمانی)، دنباله‌ی زیر ساخته می‌شود:

$$[x, t] \xrightarrow{\text{Generator}} \{[x, t_0], [x, t_1], \dots, [x, t_{k-1}]\}, \quad t_j = t + j \Delta t. \quad (5)$$

¹⁶Collocation Points

¹⁷Pseudo Sequence Generator

توجه شود که مقدار مکانی x در تمام گام‌های دنباله ثابت می‌ماند و تنها مؤلفه‌ی زمانی تغییر می‌کند. شکل ۲ این عملیات را برای $k = 5$ نشان می‌دهد.



شکل ۲: مولد دنباله‌ی شبه‌زمانی. یک ورودی نقطه‌ای (x, t) با تکرار مقدار مکانی x و افزایش تدریجی زمان با گام ثابت Δt ، به یک دنباله‌ی k -تایی تبدیل می‌شود.

انتخاب k و Δt اهمیت عملی زیادی دارد، چون این دو پارامتر مشخص می‌کنند مدل تا چه افق زمانی $(k\Delta t)$ را می‌تواند مستقیم ببیند. همان‌طور که در فصل ۵ با شواهد تجربی نشان می‌دهیم، اگر این افق با مقیاس زمانی واقعی دینامیک معادله هماهنگ نباشد، می‌تواند مستقیماً روی موفقیت یا شکست مدل اثر بگذارد.

۴.۳ گام دوم: اختلاط فضایی-زمانی^{۱۸}

هر عضو این دنباله هنوز به صورت یک بردار دوبعدی مانند $[x, t_j]$ است. این نمایش کم‌بعدی برای یادگیری روابط پیچیده توسط مکانیزم توجه کافی نیست؛ به همین دلیل، مانند معماری ترنسفورمر، ابتدا هر ورودی به یک فضای برداری با ابعاد بالاتر^{۱۹} نگاشت می‌شود تا شبکه بتواند ویژگی‌های پیچیده‌تر موجود در داده را بهتر نمایش و پردازش کند.

فرمول‌بندی. این کار با یک لایه‌ی خطی ساده انجام می‌شود:

$$\mathbf{h}_j = W_{\text{mix}} [x, t_j] + \mathbf{b}_{\text{mix}}, \quad \mathbf{h}_j \in \mathbb{R}^{d_{\text{model}}}, \quad (6)$$

که W_{mix} و $\mathbf{b}_{\text{mix}}$ پارامترهای یادگیرنده‌اند و d_{model} (در این پروژه برابر 32) بعد فضای جدید است. با این کار، دنباله‌ی k -تایی از بردارهای دوبعدی به دنباله‌ای از بردارهای d_{model} -بعدی تبدیل می‌شود که ورودی مرحله‌ی بعدی، یعنی رمزگذار-رمزگشا، خواهد بود.

۵.۳ گام سوم: رمزگذار-رمزگشای مبتنی بر مکانیزم توجه

این بخش هسته‌ی اصلی معماری است و وظیفه‌ی آن یادگیری وابستگی میان اعضای دنباله است. در این معماری، از مکانیزم خودتوجهی^{۲۰} برای تعیین میزان ارتباط هر عضو با سایر اعضای دنباله استفاده می‌شود.

۱.۵.۳ مکانیزم خودتوجهی

فرض کنید دنباله‌ای شامل k بردار داریم. در مکانیزم خودتوجهی، بازنمایی هر بردار با استفاده از اطلاعات تمام اعضای دنباله به‌روزرسانی می‌شود؛ برای هر عضو سه بردار پرسش^{۲۱}، کلید^{۲۲} و مقدار^{۲۳} ساخته می‌شود و میزان

¹⁸Spatio-Temporal Mixing

¹⁹Embedding

²⁰Self-Attention

²¹Query

²²Key

²³Value

شباهت پرسش هر عضو با کلید سایر اعضا، وزن ترکیب بردارهای مقدار را تعیین می‌کند. در پیاده‌سازی این پروژه، این محاسبه برای دنباله‌ای با طول ثابت $k = 5$ انجام می‌شود که هر عضو آن به یک بردار 32-بعدی نگاشت شده است.

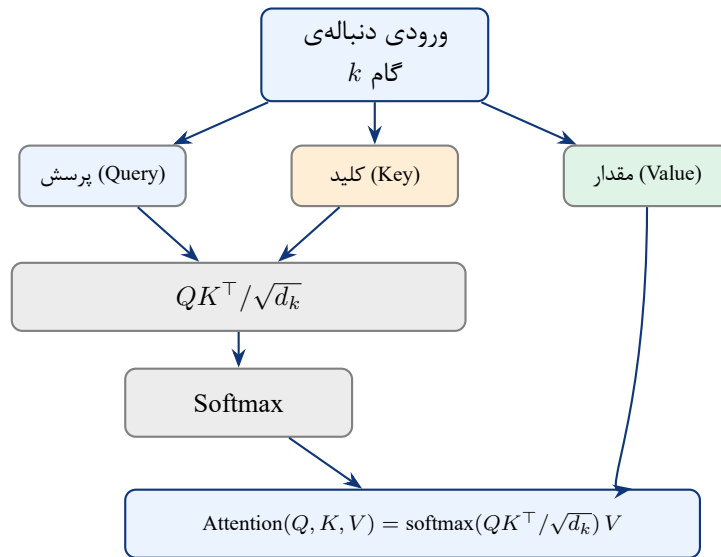
فرمول‌بندی. برای ماتریس ورودی دنباله $H \in \mathbb{R}^{k \times d_{\text{model}}}$ (که هر سطر آن یکی از بردارهای $\mathbf{h}_j$ است)، سه ماتریس پرسش، کلید، و مقدار با نگاشت‌های خطی مجزا ساخته می‌شوند:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (7)$$

و خروجی توجه به شکل زیر محاسبه می‌شود:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (8)$$

که در آن QK^T ماتریس شباهت میان تمام جفت‌های پرسش-کلید را محاسبه می‌کند، تقسیم بر $\sqrt{d_k}$ برای پایداری عددی صورت می‌گیرد، و softmax این شباهت‌ها را به وزن‌هایی با مجموع 1 تبدیل می‌کند. شکل ۳ این جریان محاسباتی را به شکل شماتیک نشان می‌دهد.



شکل ۳: مکانیزم خودتوجهی.

در عمل، مکانیزم خودتوجهی به صورت هم‌زمان در چند سر مستقل اجرا می‌شود که به آن توجه چندسر^{۲۴} گفته می‌شود. هر سر می‌تواند نوع متفاوتی از وابستگی میان اعضای دنباله را یاد بگیرد و در پایان، خروجی سرها با یکدیگر ترکیب می‌شوند. در این پروژه، برای این بخش از معماری 2 سر توجه استفاده شده است.

۲.۵.۳ ساختار رمزگذار-رمزگشا

در معماری ترنسفورمر، رمزگذار^{۲۵} دنباله‌ی ورودی را به یک بازنمایی غنی‌تر تبدیل می‌کند و رمزگشا^{۲۶} با استفاده از این بازنمایی، خروجی را تولید می‌کند. در PINN، هدف پیش‌بینی گام زمانی بعدی نیست؛ بلکه شبکه باید

²⁴Multi-Head Attention

²⁵Encoder

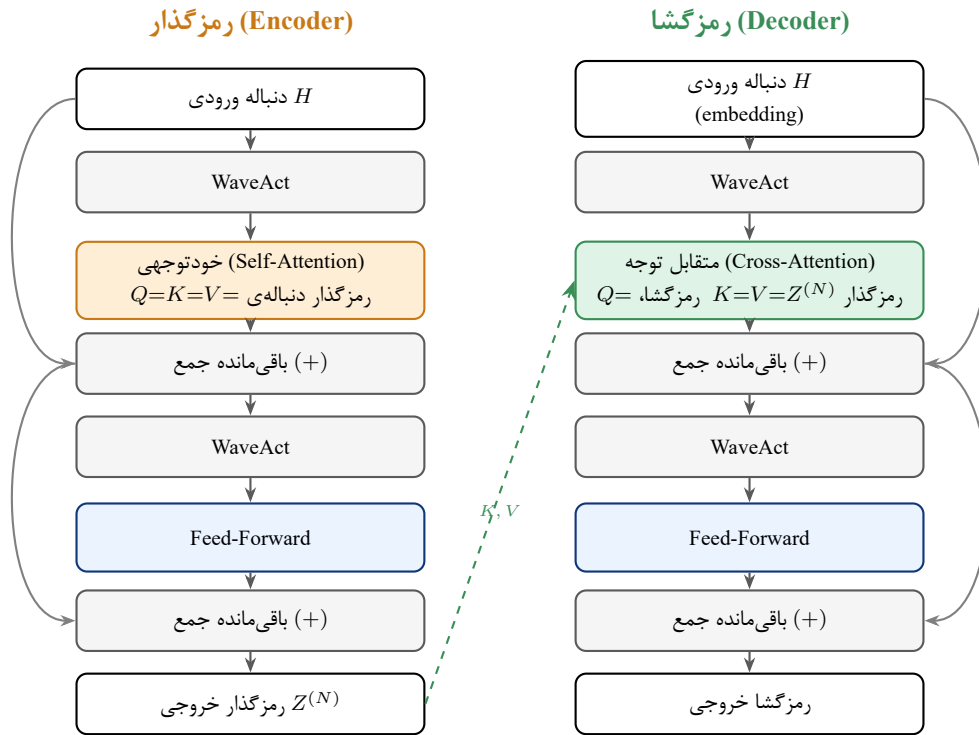
²⁶Decoder

مقدار جواب را در نقاط فضازمانی موردنظر تقریب بزند. بنابراین، ساختار رمزگشای PINNsFormer با ترنسفورمر استاندارد متفاوت است. در این ساختار، رمزگشا مستقیماً از embedding ورودی استفاده می‌کند و با یک لایه‌ی توجه رمزگذار-رمزگشا، ارتباط میان نمایش ورودی و خروجی رمزگذار را محاسبه می‌کند. سپس یک لایه‌ی پیش‌خور^{۲۷} این بازنمایی را پردازش کرده و خروجی نهایی را تولید می‌کند.

فرمول‌بندی. خروجی رمزگذار با N لایه‌ی متوالی (در این پروژه، مطابق مقاله، $N = 1$) به شکل زیر محاسبه می‌شود:

$$Z^{(0)} = H, \quad Z^{(l)} = \text{FFN}\left(Z^{(l-1)} + \text{Attention}\left(Z^{(l-1)}, Z^{(l-1)}, Z^{(l-1)}\right)\right), \quad l = 1, \dots, N, \quad (9)$$

که در آن FFN یک شبکه‌ی Feed-Forward دولایه است. در بخش رمزگشا، بردارهای Query از نمایش ورودی رمزگشا و بردارهای Key و Value از خروجی نهایی رمزگذار، یعنی $Z^{(N)}$ ، محاسبه می‌شوند. سپس خروجی این لایه‌ی توجه وارد FFN شده و پس از پردازش، به‌عنوان نمایش نهایی رمزگشا مورد استفاده قرار می‌گیرد.



شکل ۴: ساختار دقیق رمزگذار و رمزگشا در PINNsFormer

۳.۵.۳ تابع فعال‌ساز Wavelet

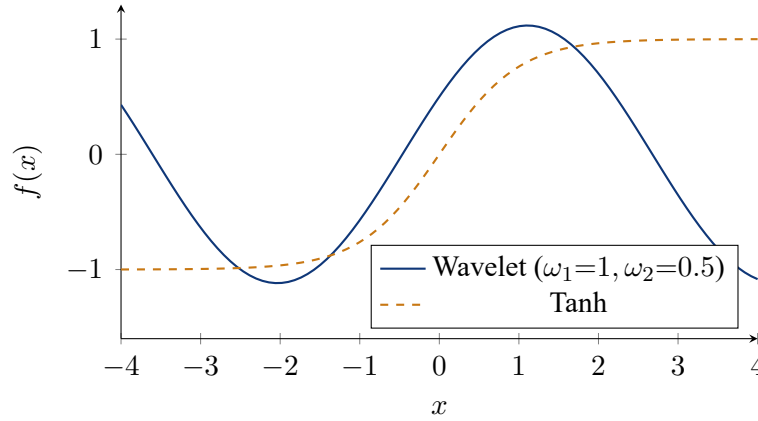
فعال‌سازهایی مانند ReLU برای PINN مناسب نیستند، زیرا مشتق آن‌ها در نقطه‌ی صفر پیوسته نیست و در محاسبه‌ی مشتقات مرتبه‌ی بالای شبکه می‌تواند مشکل ایجاد کند. فعال‌ساز Tanh از این نظر هموار است و مشتقات مرتبه‌ی بالا دارد، اما خروجی آن به بازه‌ی $[-1, 1]$ محدود می‌شود و ممکن است در تقریب توابع نوسانی یا با دامنه‌ی بزرگ محدودیت ایجاد کند. نویسندگان مقاله‌ی اصلی [۱] برای رفع این محدودیت فعال‌سازی با نام Wavelet پیشنهاد داده‌اند که با الهام از نمایش فوریه طراحی شده است.

²⁷Feed-Forward

فرمول‌بندی. این فعال‌ساز به شکل زیر تعریف می‌شود:

$$\text{Wavelet}(x) = \omega_1 \sin(x) + \omega_2 \cos(x), \quad (10)$$

که ω_1, ω_2 پارامترهای یادگیرنده (نه ثابت) هستند و به شبکه اجازه می‌دهند دامنه و ترکیب سینوس/کسینوس را متناسب با هر مسئله تنظیم کند. شکل ۵ شکل این تابع را (برای مقادیر نمونه‌ی $\omega_1 = 1, \omega_2 = 0.5$) در مقایسه با Tanh نشان می‌دهد.



شکل ۵: مقایسه‌ی شکل تابع فعال‌ساز Wavelet با Tanh. برخلاف Tanh که به شکل یکنوا به مقادیر حدی ± 1 میل می‌کند، Wavelet رفتاری نوسانی و غیریکنوا دارد که می‌تواند برای تقریب توابع نوسانی مناسب‌تر باشد.

ویژگی نظری: تقریب‌زنی جهانی. این خاصیت، یعنی توانایی یک شبکه‌ی دولایه با عرض کافی و فعال‌ساز Wavelet در تقریب هر تابع حقیقی مقدار با دقت دلخواه، در مقاله‌ی اصلی [۱] با استفاده از نمایش فوریه‌ی حقیقی اثبات شده است؛ ما این نتیجه را از مقاله نقل می‌کنیم و در این پروژه اثبات مستقلی برای آن ارائه نکرده‌ایم. ایده‌ی کلی اثبات بر نمایش فوریه‌ای تابع و تقریب انتگرال مربوطه با یک مجموع متناهی استوار است.

۶.۳ گام چهارم: لایه‌ی خروجی (Output Layer)

پس از پردازش دنباله توسط رمزگذار و رمزگشا، هر عضو دنباله یک بردار d_{model} -بعدی است. لایه‌ی خروجی این بردار را به مقدار تقریبی جواب PDE نگاشت می‌کند.

فرمول‌بندی. لایه‌ی خروجی یک شبکه‌ی MLP است که به‌صورت مستقل روی هر عضو دنباله اعمال می‌شود:

$$\hat{u}(x, t_j) = \text{MLP}(\mathbf{z}_j), \quad j = 0, \dots, k-1, \quad (11)$$

که در آن $\mathbf{z}_j$ عضو j -ام خروجی رمزگشا است. به این ترتیب شبکه برای هر یک از k زمان موجود در دنباله، یک مقدار تقریبی تولید می‌کند. مقدار اول، یعنی $\hat{u}(x, t_0) = \hat{u}(x, t)$ ، پاسخ موردنظر در نقطه‌ی ورودی اصلی است.

۷.۳ تعداد پارامترها و پیچیدگی محاسباتی

تعداد پارامترها. با تنظیمات استفاده‌شده در این پروژه (جدول ۱)، تعداد پارامترهای قابل‌یادگیری دو مدل در یک مقیاس قرار دارد. مدل PINNsFormer با $k = 5$ ، $d_{\text{model}} = 32$ و $d_{\text{hidden}} = 512$ حدود 454,000 پارامتر و PINN استاندارد با چهار لایه‌ی پنهان 512-بعدی حدود 527,000 پارامتر دارد. نزدیک بودن این تعداد باعث می‌شود مقایسه‌ی دو روش بیشتر بر تفاوت معماری آن‌ها متمرکز باشد تا تفاوت در ظرفیت شبکه.

پیچیدگی محاسباتی. در PINNsFormer، هر نقطه‌ی ورودی به یک دنباله‌ی k -تایی تبدیل می‌شود و به همین دلیل تعداد پردازش‌های شبکه نسبت به PINN استاندارد افزایش می‌یابد. همچنین، مکانیزم خودتوجهی برای دنباله‌ای با طول k دارای هزینه‌ی محاسباتی $O(k^2 d_{\text{model}})$ است. از آنجا که در این پروژه $k = 5$ انتخاب شده است، این هزینه محدود باقی می‌ماند؛ اما چون افزایش k باعث رشد درجه دوم هزینه‌ی خودتوجهی می‌شود، انتخاب طول کوتاه برای دنباله از نظر محاسباتی اهمیت دارد.

۸.۳ تابع هزینه‌ی دنباله‌ای

از آنجا که خروجی PINNsFormer به صورت یک دنباله تولید می‌شود، تابع هزینه‌ی PINN نیز باید متناسب با این ساختار تعریف شود. برای تمام اعضای دنباله، یعنی نقاط (x, t_j) ، می‌توان شرط معادله و شرایط مرزی را اعمال کرد؛ زیرا این نقاط در دامنه یا روی مرز مسئله قرار دارند. اما شرط اولیه فقط در $t = 0$ تعریف شده است، بنابراین تنها به اولین عضو دنباله اعمال می‌شود.

فرمول‌بندی. بر همین اساس، سه جمله‌ی تابع هزینه به شکل زیر تعریف می‌شوند:

$$\mathcal{L}_{res} = \frac{1}{kN_{res}} \sum_{i=1}^{N_{res}} \sum_{j=0}^{k-1} \|\mathcal{D}[\hat{u}(x_i, t_j)] - f(x_i, t_j)\|^2, \quad (12)$$

$$\mathcal{L}_{bc} = \frac{1}{kN_{bc}} \sum_{i=1}^{N_{bc}} \sum_{j=0}^{k-1} \|\mathcal{B}[\hat{u}(x_i, t_j)] - g(x_i, t_j)\|^2, \quad (13)$$

$$\mathcal{L}_{ic} = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \|\hat{u}(x_i, t_0) - h(x_i)\|^2, \quad (14)$$

و تابع هزینه‌ی نهایی، مجموع وزن دار این سه جمله است:

$$\mathcal{L}_{PINNsFormer} = \lambda_{res}\mathcal{L}_{res} + \lambda_{ic}\mathcal{L}_{ic} + \lambda_{bc}\mathcal{L}_{bc}. \quad (15)$$

تمام k گام دنباله در محاسبه‌ی $\mathcal{L}_{res}$ و $\mathcal{L}_{bc}$ سهیم‌اند، اما تنها گام نخست، که دقیقاً منطبق بر $t = 0$ است، در محاسبه‌ی $\mathcal{L}_{ic}$ استفاده می‌شود.

در تمام آزمایش‌های این پروژه، $\lambda_{res} = \lambda_{ic} = \lambda_{bc} = 1$ در نظر گرفته شده است.

۹.۳ الگوریتم آموزش

با کنار هم گذاشتن معماری (بخش‌های ۳.۳ تا ۹.۳) و تابع هزینه‌ی دنباله‌ای (بخش ۸.۳)، الگوریتم ۱ کل روند آموزش، از آماده‌سازی نقاط کولوکیشن تا همگرایی، را به شکل شبه‌کد خلاصه می‌کند. این الگوریتم دقیقاً همان کدی است که در فصل‌های بعدی این گزارش برای هر معادله اجرا شده است.

الگوریتم ۱ Training PINNsFormer

```

Initialize parameters  $\theta$  using the Xavier Uniform method
Sample collocation points  $\{(x_i, t_i)\}_{i=1}^{N_r}, \{(x_i, t_i)\}_{i=1}^{N_b}, \{x_i\}_{i=1}^{N_{ic}}$ 
Expand each point via the pseudo-sequence generator (Section 3.3) into a  $k$ -tuple sequence
while not converged and iter < itermax do
     $\hat{u}(x_i, t_j) \leftarrow \text{PINNsFormer}_{\theta}(x_i, t_j)$  for all points and  $j = 0, \dots, k - 1$ 
    Compute the required derivatives ( $u_t, u_x, u_{xx}, \dots$ ) via automatic differentiation
    Compute  $\mathcal{L}_{res}, \mathcal{L}_{bc}, \mathcal{L}_{ic}$  according to Eqs. (9)–(11)
     $\mathcal{L} \leftarrow \lambda_{res}\mathcal{L}_{res} + \lambda_{ic}\mathcal{L}_{ic} + \lambda_{bc}\mathcal{L}_{bc}$ 
     $\theta \leftarrow \theta - \text{L-BFGS-step}(\nabla_{\theta}\mathcal{L})$  ▷ with strong Wolfe line search
    iter  $\leftarrow$  iter + 1
end while
return  $\theta$ 
    
```

نکته‌ی مهم سطر ۵ این الگوریتم این است که مدل برای تمام k گام دنباله فراخوانی می‌شود، نه فقط گام نخست؛ چون محاسبه‌ی $\mathcal{L}_{res}$ و $\mathcal{L}_{bc}$ برخلاف $\mathcal{L}_{ic}$ ، به تمام گام‌ها نیاز دارد. الگوریتم معادل برای PINN استاندارد (بخش ۹؟) همین ساختار کلی را دارد، با این تفاوت که سطر ۳ (بسط دنباله) در آن نیست و $\mathcal{L}_{res}$ و $\mathcal{L}_{bc}$ به شکل نقطه‌ای، نه دنباله‌ای، محاسبه می‌شوند.

۱۰.۳ مقایسه‌ی PINN و PINNsFormer

جدول ۱ تفاوت‌های معماری این دو مدل، که پیش‌تر در این فصل به تفصیل شرح داده شد، را در یک نگاه خلاصه می‌کند.

جدول ۱: مقایسه‌ی ساختاری PINN استاندارد و PINNsFormer

PINNsFormer	PINN استاندارد	ویژگی
دنباله‌ی k -تایی	نقطه‌ی تنها (x, t)	واحد پردازش ورودی
خودتوجهی (self-attention)	ندارد	مکانیزم یادگیری وابستگی زمانی
Wavelet	Tanh	تابع فعال‌ساز
دنباله‌ای (میانگین روی k گام)	نقطه‌ای	ساختار شرط اولیه/مرزی در loss
~ 454000	~ 527000	تعداد پارامتر (تقریبی)
$\sim 2.9 \times$	$1 \times$ (مبنا)	سربار محاسباتی نسبی ($k=5$)

۱۱.۳ معیارهای ارزیابی

برای سنجش کمی دقت هر مدل نسبت به جواب مرجع u ، از دو معیار خطای نسبی استاندارد PINN استفاده شده است:

$$\text{rMAE} = \frac{\sum_{n=1}^N |\hat{u}(x_n, t_n) - u(x_n, t_n)|}{\sum_{n=1}^N |u(x_n, t_n)|}, \quad (16)$$

$$\text{rRMSE} = \sqrt{\frac{\sum_{n=1}^N |\hat{u}(x_n, t_n) - u(x_n, t_n)|^2}{\sum_{n=1}^N |u(x_n, t_n)|^2}}, \quad (17)$$

که N تعداد نقاط گزیده ارزیابی است. هر دو معیار عددی در بازه $[0, \infty)$ می‌دهند؛ هرچه این عدد به صفر نزدیک‌تر باشد، مدل دقیق‌تر است. در عمل، مقدار نزدیک به 1 معمولاً نشانه‌ی آن است که مدل به همگرایی مناسبی نرسیده و خروجی آن با جواب واقعی همبستگی کمی دارد؛ هرچند این آستانه دقیق و مطلق نیست و باید در کنار شکل نمودار خروجی تفسیر شود.

۱۲.۳ محدودیت‌های معماری PINNsFormer

وابستگی به ابرپارامترهای k و Δt . در مولد دنباله‌ی شبه‌زمانی، طول دنباله k و فاصله‌ی زمانی Δt از پیش تعیین می‌شوند و افق زمانی مدل را مشخص می‌کنند. بنابراین، یک انتخاب ثابت از این پارامترها لزوماً برای مسائل با مقیاس‌های زمانی متفاوت مناسب نیست و تنظیم آن‌ها به ویژگی‌های مسئله وابسته است.

هزینه‌ی حافظه برای مشتقات مرتبه‌ی بالا. محاسبه‌ی مشتقات مرتبه‌ی بالا با Autograd نیازمند نگهداری گراف محاسباتی در هر مرحله‌ی مشتق‌گیری است. این هزینه در PINNsFormer به دلیل وجود لایه‌های توجه و پردازش دنباله‌ای بیشتر است و در مسائل دارای مشتقات مرتبه‌ی بالا می‌تواند محدودیت حافظه ایجاد کند. در این پروژه نیز همین مسئله موجب کاهش تعداد نقاط کولوکیشن در معادله‌ی دارای مشتق مرتبه‌ی چهارم شد.

عدم محدودیت صریح خروجی فعال‌ساز Wavelet. برخلاف Tanh که خروجی آن در بازه‌ی $[-1, 1]$ قرار دارد، پارامترهای فعال‌ساز Wavelet محدودیت صریحی ندارند و ممکن است در طول آموزش مقادیر بزرگی پیدا کنند. این ویژگی انعطاف‌پذیری بیشتری ایجاد می‌کند، اما می‌تواند احتمال ناپایداری عددی را، به‌ویژه در فرایند بهینه‌سازی، افزایش دهد.

۴ معادلات مورد بررسی و نتایج

۱.۴ معادله‌ی برگرز (Burgers' Equation) [۱۴]

معادله‌ی برگرز لزجت‌دار ^{۲۸} یکی از مسائل کلاسیک در معادلات دیفرانسیل غیرخطی است. این معادله به دلیل ایجاد ناحیه‌ای با تغییرات سریع و گرادیان‌های بزرگ ^{۲۹} در نزدیکی $x = 0$ ، به‌ویژه برای زمان‌های بزرگ، مسئله‌ای چالش‌برانگیز برای روش‌های عددی و یادگیری عمیق به شمار می‌رود:

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \frac{0.01}{\pi} \frac{\partial^2 u}{\partial x^2} = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = -\sin(\pi x), \\ u(-1, t) = u(1, t) = 0. \end{cases} \quad (18)$$

^{۲۸}Viscosity^{۲۹}Near-Shock

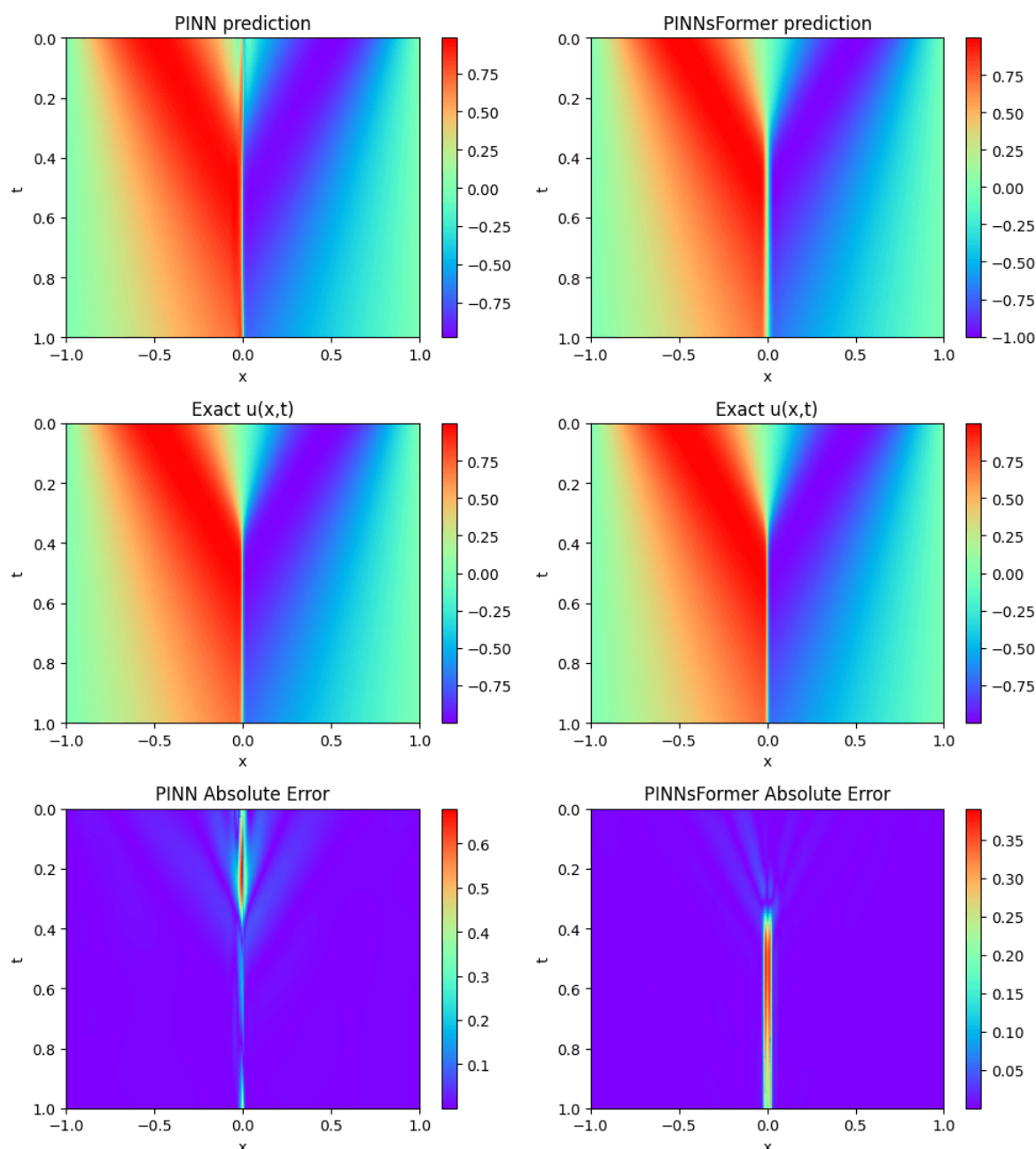
تنظیمات این آزمایش در جدول زیر آمده است:

جدول ۲: تنظیمات آزمایش معادله‌ی برگرز	
51×51	تراکم نقاط collocation ($N_x \times N_t$)
500	تعداد iteration آموزش
منبع جواب مرجع burgers_shock.mat (Raissi et al., 2019)	

۱.۱.۴ نتایج و تحلیل

جدول ۳: نتایج نهایی loss و خطای نسبی - معادله‌ی برگرز			
مدل	rMAE	rRMSE	زمان آموزش
PINN استاندارد	0.0262	0.0695	4 دقیقه و 1 ثانیه
PINNsFormer	0.0120	0.0556	28 دقیقه و 25 ثانیه

مقادیر نهایی اجزای تابع هزینه: PINN استاندارد به $\mathcal{L}_{res} = 1.03 \times 10^{-3}$ و PINNsFormer به $\mathcal{L}_{res} = 9.07 \times 10^{-5}$ رسیدند؛ یعنی PINNsFormer باقی‌مانده‌ی معادله را حدود یک مرتبه‌ی بزرگی بهتر برآورده کرده است.



شکل ۶: مقایسه‌ی پیش‌بینی PINN و PINNsFormer با جواب مرجع، به همراه خطای مطلق^{۳۰} معادله‌ی برگرز.

مقدار rMAE با استفاده از PINNsFormer از 0.0262 به 0.0120 کاهش یافت، یعنی حدود 54% بهبود نسبت به PINN استاندارد. با این حال PINN استاندارد در این معادله هم عملکرد قابل‌قبولی داشت و دچار شکست جدی نشد؛ رقم خطای آن (0.0262) در محدوده‌ای است که هنوز تقریب معقولی از جواب به شمار می‌رود. به همین دلیل، بهبود مشاهده‌شده در معادله‌ی برگرز را بیشتر باید افزایش تدریجی دقت دانست تا رفع یک حالت شکست.

³⁰absolute error

۲.۴ معادله‌ی آلن-کان (Allen-Cahn Equation) [۱۵]

معادله‌ی آلن-کان به دلیل ضریب نفوذ کوچک ^{۳۱} و وجود یک جمله‌ی واکنشی غیرخطی ^{۳۲}، یکی از مسائل چالش‌برانگیز برای روش‌های PINN است. همچنین، این معادله دارای دو حالت پایدار $u = \pm 1$ است که می‌توانند در فرایند آموزش با یکدیگر رقابت کنند و شبکه را به سمت کمینه‌های محلی ^{۳۳} هدایت کنند.

$$\begin{cases} \frac{\partial u}{\partial t} - 0.0001 \frac{\partial^2 u}{\partial x^2} + 5u^3 - 5u = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = x^2 \cos(\pi x), \\ u(-1, t) = u(1, t), \\ u_x(-1, t) = u_x(1, t). \end{cases} \quad (۱۹)$$

جدول ۴: تنظیمات آزمایش معادله‌ی آلن-کان

51×51	تراکم نقاط collocation $(N_x \times N_t)$
500	تعداد iteration آموزش
(Raissi et al., 2019) AC.mat	منبع جواب مرجع

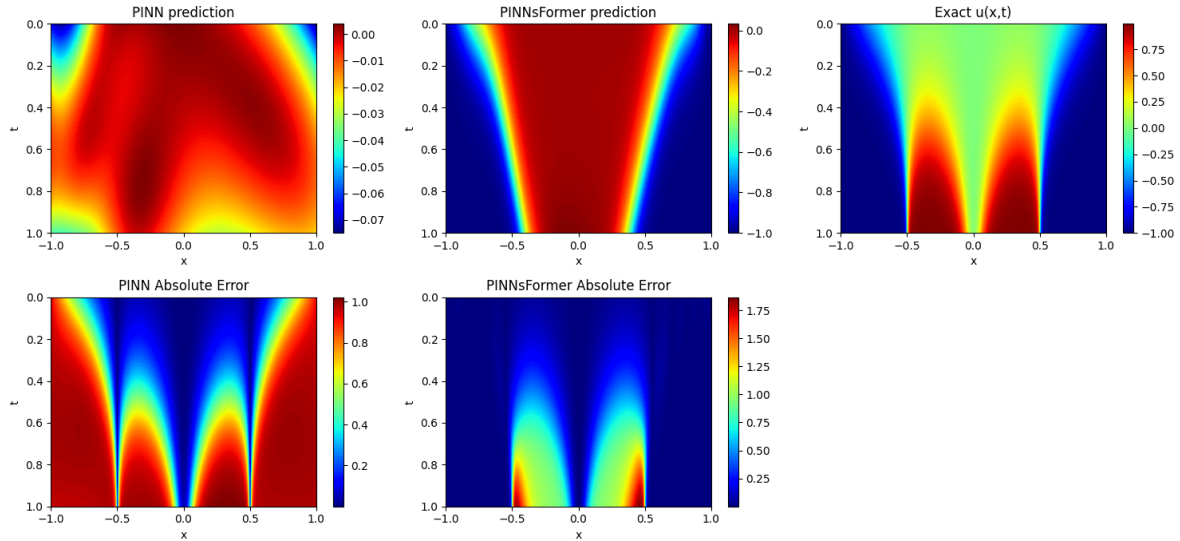
۱.۲.۴ نتایج و تحلیل

جدول ۵: نتایج نهایی loss و خطای نسبی - معادله‌ی آلن-کان

مدل	rMAE	rRMSE	زمان آموزش
PINN استاندارد	0.9916	0.9910	1 دقیقه و 28 ثانیه
PINNsFormer	0.3790	0.6100	32 دقیقه و 53 ثانیه

مقادیر نهایی اجزای تابع هزینه: PINN استاندارد به $\mathcal{L}_{res} = 1.15 \times 10^{-2}$ و $\mathcal{L}_{ic} = 1.36 \times 10^{-1}$ رسید؛ خطای شرط اولیه تقریباً یک مرتبه‌ی بزرگی بیشتر از خطای باقی‌مانده است، که نشان می‌دهد مدل حتی در برازش شرط اولیه هم به مشکل خورده. PINNsFormer در همین دو جمله به $\mathcal{L}_{res} = 1.12 \times 10^{-4}$ و $\mathcal{L}_{ic} = 6.78 \times 10^{-4}$ رسید، کاهش حدود دو مرتبه‌ی بزرگی در هر دو، که با کاهش 62% در rMAE هم‌خوانی دارد.

³¹Diffusion Coefficient³²Stiff³³Local Minima



شکل ۷: مقایسه‌ی پیش‌بینی PINN و PINNsFormer با جواب مرجع - معادله‌ی آلن-کان. توجه شود که PINN استاندارد هیچ ساختار زمانی معناداری یاد نگرفته، در حالی که PINNsFormer شکل هندسی رابطها را تا حد زیادی بازتولید کرده است.

این معادله در میان پنج مسئله‌ی بررسی‌شده، بیشترین چالش را برای PINN استاندارد ایجاد کرد؛ مقدار $rMAE \approx 0.99$ عملاً یعنی مدل نتوانسته جواب معادله را تقریب بزند. PINNsFormer این عدد را به $rMAE \approx 0.38$ رساند، کاهشی حدود 62% و توانست بخش قابل‌توجهی از ساختار رابطهای متحرک (moving interfaces) را بازسازی کند، هرچند در برخی نواحی، در تشخیص حالت پایدار مناسب بین $+1$ و -1 دچار خطا شد. در این معادله، مدل‌سازی وابستگی‌های زمانی خطا را به‌طور محسوسی کاهش داد، اما $rMAE \approx 0.38$ همچنان خطای قابل‌توجهی است و نمی‌توان گفت مسئله به‌طور کامل حل شد.

۳.۴ معادله‌ی شرودینگر غیرخطی (Nonlinear Schrödinger Equation) [۱۶]

معادله‌ی شرودینگر غیرخطی یکی از بنچمارک‌های اصلی در مطالعات اولیه‌ی PINN است. از آنجا که جواب معادله مختلط است، به‌صورت $h = u + iv$ نمایش داده می‌شود. بنابراین، خروجی شبکه شامل دو مؤلفه‌ی مستقل، یعنی بخش حقیقی^{۳۴} و بخش موهومی^{۳۵} جواب، در نظر گرفته شده است:

$$\begin{cases} ih_t + 0.5h_{xx} + |h|^2h = 0, & x \in [-5, 5], t \in [0, \pi/2], \\ h(x, 0) = 2 \operatorname{sech}(x), \\ h(-5, t) = h(5, t), \\ h_x(-5, t) = h_x(5, t). \end{cases} \quad (20)$$

با تفکیک بخش حقیقی و موهومی، دو معادله‌ی حقیقی زیر برای باقی‌مانده به‌دست می‌آید:

$$f_u \equiv u_t + 0.5v_{xx} + (u^2 + v^2)v = 0, \quad f_v \equiv v_t - 0.5u_{xx} - (u^2 + v^2)u = 0$$

^{۳۴}Real

^{۳۵}Imaginary

جدول ۶: تنظیمات آزمایش معادله‌ی شرودینگر

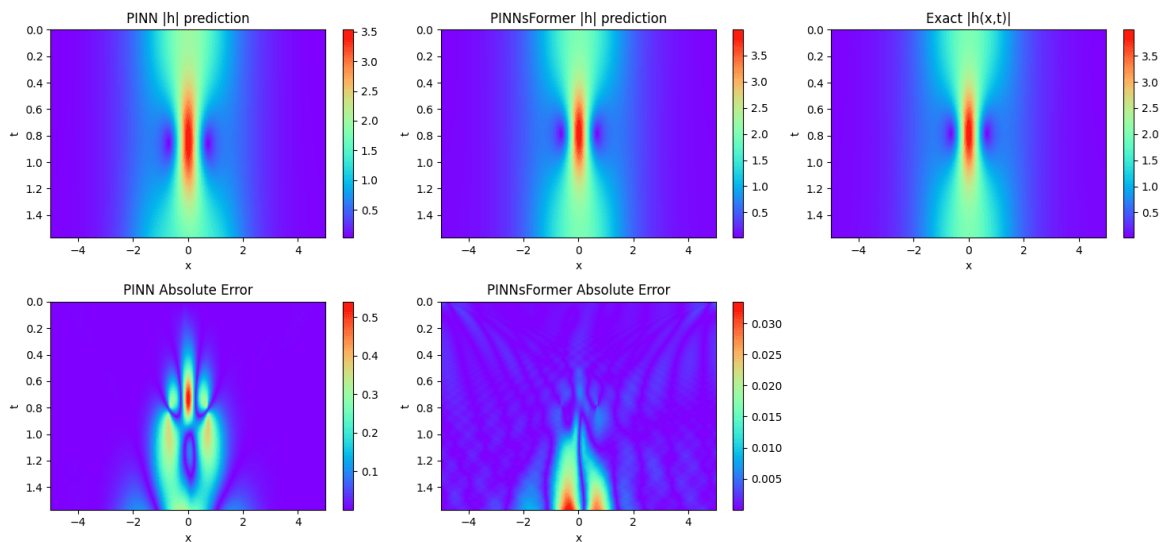
تراکم نقاط collocation $(N_x \times N_t)$	51×51
تعداد iteration آموزش	500
منبع جواب مرجع	(Raissi et al., 2019) NLS.mat
معیار ارزیابی	دامنه‌ی جواب مختلط $ h = \sqrt{u^2 + v^2}$

۱.۳.۴ نتایج و تحلیل

جدول ۷: نتایج نهایی loss و خطای نسبی - معادله‌ی شرودینگر

مدل	rMAE	rRMSE	زمان آموزش
PINN استاندارد	0.0601	0.0926	7 دقیقه و 12 ثانیه
PINNsFormer	0.0032	0.0047	49 دقیقه و 59 ثانیه

مقادیر نهایی اجزای تابع هزینه: برخلاف بیشتر معادلات دیگر، اینجا باقی‌مانده‌ی معادله در PINN استاندارد $(\mathcal{L}_{res} = 1.07 \times 10^{-5})$ کمی کمتر از مقدار متناظر در PINNsFormer $(\mathcal{L}_{res} = 1.14 \times 10^{-5})$ به دست آمد. پس بهبود قابل توجه rMAE در این معادله را نمی‌توان به کاهش باقی‌مانده‌ی معادله نسبت داد؛ بخش مهم‌تری از آن به کاهش خطای شرط اولیه (از $\mathcal{L}_{ic} = 5.03 \times 10^{-6}$ به 1.77×10^{-6}) و توزیع یکنواخت‌تر خطا در سراسر دامنه برمی‌گردد.

شکل ۸: مقایسه‌ی پیش‌بینی PINN و PINNsFormer با جواب مرجع روی دامنه‌ی $|h(x, t)|$ - معادله‌ی شرودینگر.

معادله‌ی شرودینگر بهترین نتیجه‌ی کمی پروژه را برای PINNsFormer داشت: کاهش rMAE از 0.0601 به 0.0032، یعنی حدود 95%. هر دو مدل توانستند شکل کلی سالیتون^{۳۶} را بازسازی کنند، ولی PINNsFormer در

³⁶Soliton

نواحی با تغییرات سریع دامنه دقت بیشتری داشت، شاهد دیگری بر اینکه مکانیزم Self-Attention در دینامیک زمانی غیرخطی و پیوسته کارآمد است.

۴.۴ معادله‌ی کوراموتو-سیواشینسکی (Kuramoto-Sivashinsky Equation) [۱۷]

معادله‌ی کوراموتو-سیواشینسکی یک معادله‌ی دیفرانسیل جزئی مرتبه‌ی چهارم است که برای توصیف پدیده‌های آشوبناک^{۳۷} و ناپایداری‌های جبهه‌ی شعله^{۳۸} استفاده می‌شود. در حالت کلی، این معادله شامل مشتقات تا مرتبه‌ی چهارم است و با صفر قرار دادن ضریب مشتق مرتبه‌ی سوم، به فرم استاندارد کوراموتو-سیواشینسکی تبدیل می‌شود. فرم کلی معادله به صورت زیر است:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \alpha \frac{\partial^2 u}{\partial x^2} + \beta \frac{\partial^3 u}{\partial x^3} + \gamma \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [-10, 10], \quad t \in [0, 1] \quad (21)$$

با شرط اولیه $u(x, 0) = h(x)$ و شرایط مرزی دیریکله^{۳۹} و نویمان^{۴۰}

$$u(a, t) = f_1(t), \quad u(b, t) = f_2(t), \quad u_x(a, t) = g_1(t), \quad u_x(b, t) = g_2(t)$$

که در آن f_1, f_2, g_1, g_2 مستقیماً از مقدار و مشتق جواب دقیق تحلیلی در مرزها گرفته می‌شوند (نه شرط تناوبی). با $\alpha = 1, \beta = 4, \gamma = 1$ ، جواب دقیق مورد استفاده یک پالس متحرک تحلیلی^{۴۱} است:

$$u(x, t) = 11 + 15 \tanh(\theta) - 15 \tanh^2(\theta) - 15 \tanh^3(\theta), \quad \theta = -\frac{1}{2}x + t$$

صحت این جواب پیش از پیاده‌سازی با محاسبات نمادین^{۴۲} تأیید شد (باقی‌مانده‌ی معادله دقیقاً صفر است).

جدول ۸: تنظیمات آزمایش معادله‌ی کوراموتو-سیواشینسکی	
تراکم نقاط $(N_x \times N_t)$ collocation	^{۴۳} 35×35
تعداد iteration آموزش	500
منبع جواب مرجع	جواب تحلیلی بسته ^{۴۴}

^{۳۷}Chaotic Phenomena

^{۳۸}Flame Front Instabilities

^{۳۹}Dirichlet

^{۴۰}Neumann

^{۴۱}Analytical Moving Pulse

^{۴۲}Symbolic Computation

^{۴۳}به‌دلیل محدودیت حافظه‌ی GPU ناشی از عمق گراف محاسباتی مشتق مرتبه‌ی چهارم، این تراکم نسبت به سایر معادلات (که 51×51

بود) کاهش یافت.

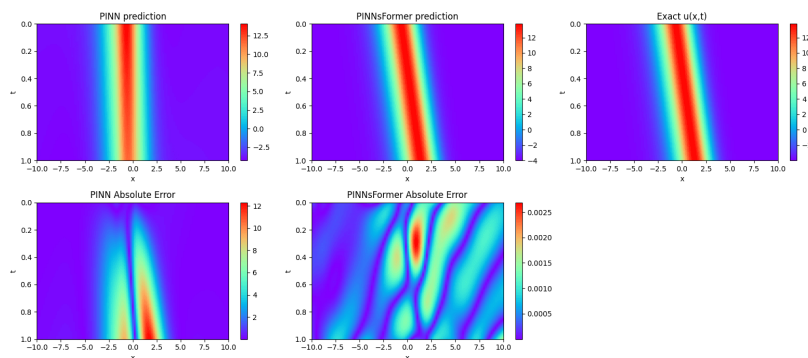
^{۴۴}Closed-Form

۱.۴.۴ نتایج و تحلیل

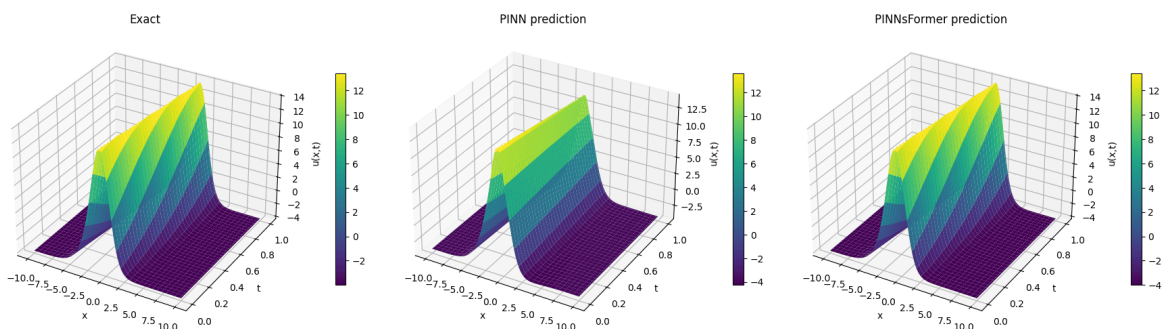
جدول ۹: نتایج نهایی loss و خطای نسبی - معادله‌ی کوراموتو-سیواشینسکی

مدل	rMAE	rRMSE	زمان آموزش
PINN استاندارد	0.3012	0.5124	2 دقیقه و 53 ثانیه
PINNsFormer	0.0001	0.0001	53 دقیقه و 57 ثانیه

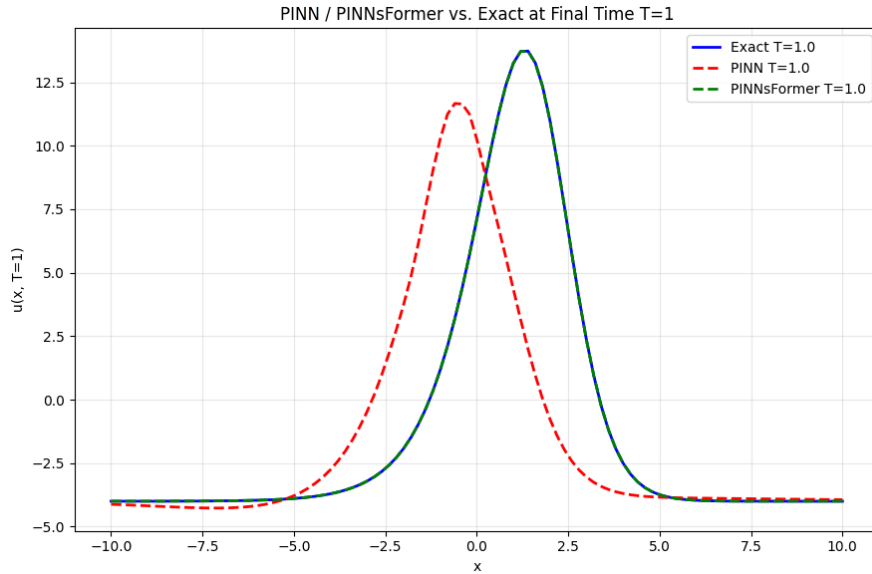
مقادیر نهایی اجزای تابع هزینه: PINN استاندارد به $\mathcal{L}_{ic} = 3.14 \times 10^{-3}$, $\mathcal{L}_{bc} = 5.53 \times 10^{-3}$, $\mathcal{L}_{res} = 4.02 \times 10^{-2}$ رسید؛ PINNsFormer به مقادیر به مراتب کوچک‌تری در هر سه جمله دست یافت ($\mathcal{L}_{ic} = 4.21 \times 10^{-7}$, $\mathcal{L}_{bc} = 6.16 \times 10^{-7}$, $\mathcal{L}_{res} = 1.35 \times 10^{-5}$).



شکل ۹: مقایسه‌ی پیش‌بینی PINN و PINNsFormer با جواب مرجع - معادله‌ی کوراموتو-سیواشینسکی. PINN استاندارد قادر به دنبال کردن حرکت پالس نیست و آن را عملاً نزدیک موقعیت اولیه ثابت نگه می‌دارد؛ PINNsFormer حرکت پالس را با خطای مطلق بسیار کوچک (حداکثر در حدود 0.0025) بازتولید کرده است.



شکل ۱۰: نمای سه‌بعدی $u(x,t)$ برای جواب دقیق، پیش‌بینی PINN، و پیش‌بینی PINNsFormer - معادله‌ی کوراموتو-سیواشینسکی.



شکل ۱۱: مقایسه‌ی جواب دقیق PINN و PINNsFormer در زمان نهایی $T = 1$. منحنی PINNsFormer عملاً بر منحنی جواب دقیق منطبق است، در حالی که PINN موقعیت و دامنه‌ی پالس را به‌درستی بازتولید نکرده است.

در معادله‌ی کوراموتو-سیواشینسکی، مقدار PINNsFormer rMAE را بیش از 99.9% کاهش داد (0.3012 در برابر 0.0001)، بزرگ‌ترین بهبود در میان پنج معادله‌ی این پروژه. در زمان نهایی $T = 1$ ، PINN استاندارد حتی موقعیت پالس را هم درست تخمین نزد: پیک پیش‌بینی‌شده نزدیک $x \approx -0.5$ بود، در حالی که موقعیت واقعی پالس در $x \approx 1.3$ قرار داشت. جواب PINNsFormer، در مقابل، تقریباً با جواب دقیق منطبق شد. این بهبود احتمالاً به انتخاب افق زمانی دنباله، $k\Delta t = 5 \times 10^{-4}$ ، مربوط است که با مقیاس زمانی حرکت پالس در این مسئله هم‌خوانی دارد و به مکانیزم Self-Attention اجازه می‌دهد وابستگی‌های زمانی مؤثر در دینامیک معادله را یاد بگیرد. به عبارت دیگر، عملکرد قوی PINNsFormer روی این معادله، دست‌کم تا حدی، وابسته به این سازگاری میان مقیاس فیزیکی مسئله و ابرپارامترهای ثابت دنباله است.

۵.۴ معادله‌ی کورتوگ-دووریس (Korteweg-de Vries Equation) [۱۸]

معادله‌ی KdV، یکی دیگر از مسئله‌های اصلی مقاله‌ی بنیادین PINN، معادله‌ای با مشتق مرتبه‌ی سوم است:

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + 0.0025 \frac{\partial^3 u}{\partial x^3} = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = \cos(\pi x) \\ u(-1, t) = u(1, t), \\ u_x(-1, t) = u_x(1, t). \end{cases} \quad (22)$$

جدول ۱۰: تنظیمات آزمایش معادله KdV

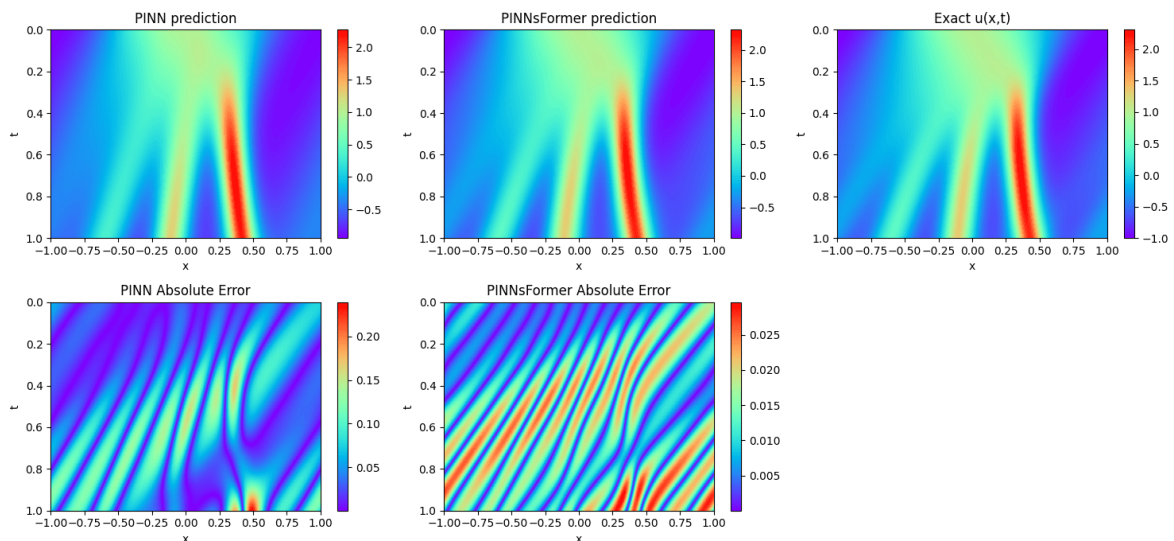
51×51	تراکم نقاط collocation ($N_x \times N_t$)
500	تعداد iteration آموزش
(Raissi et al., 2019) KdV.mat	منبع جواب مرجع

۱.۵.۴ نتایج و تحلیل

جدول ۱۱: نتایج نهایی loss و خطای نسبی - معادله KdV

مدل	rMAE	rRMSE	زمان آموزش
PINN استاندارد	0.0820	0.0857	10 دقیقه و 38 ثانیه
PINNsFormer	0.0184	0.0181	1 ساعت و 49 دقیقه

مقادیر نهایی اجزای تابع هزینه: PINN استاندارد به $\mathcal{L}_{res} = 1.55 \times 10^{-3}$ و PINNsFormer به $\mathcal{L}_{res} = 3.71 \times 10^{-5}$ رسیدند؛ کاهش به بزرگی تقریباً یک مرتبه که با بهبود مشابه در جمله شرط مرزی (3.71×10^{-5} به 4.50×10^{-6}) و شرط اولیه (1.01×10^{-3} به 4.66×10^{-5}) همراه بوده است.



شکل ۱۲: مقایسه‌ی پیش‌بینی PINN و PINNsFormer با جواب مرجع - معادله KdV.

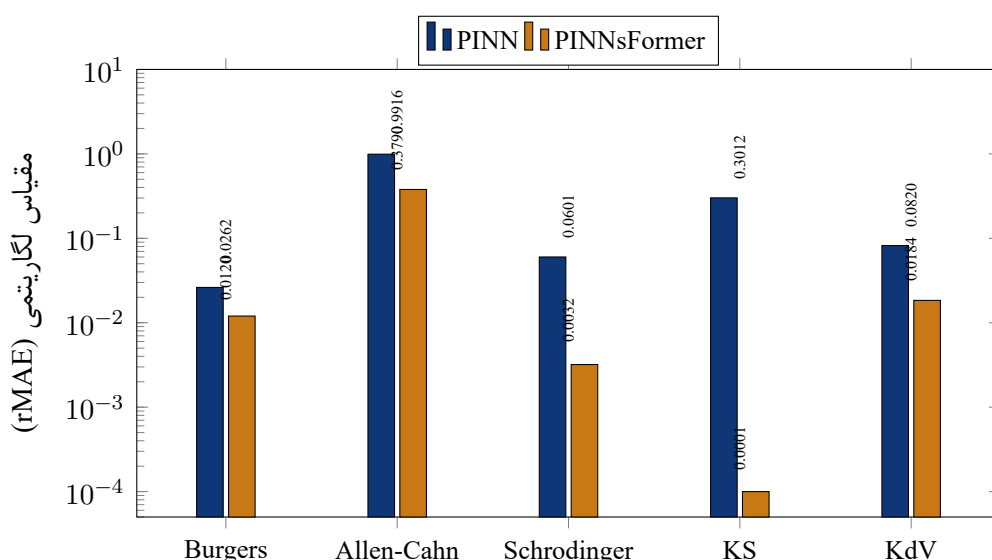
مقدار rMAE با PINNsFormer از 0.0820 به 0.0184 کاهش یافت، یعنی حدود 78% بهتر از PINN استاندارد. نکته‌ی قابل توجه این آزمایش این بود که مشتق مرتبه‌ی سوم (u_{xxx}) بدون هیچ تغییری در معماری یا فرمول‌بندی روش با موفقیت محاسبه شد؛ یعنی چارچوب، دست‌کم برای این معادله، به مشتقات مرتبه‌ی بالاتر از آنچه مقاله‌ی اصلی آزموده بود هم تعمیم پیدا کرد.

۵ تحلیل تطبیقی و بحث کلی

جدول ۱۲ نتایج کمی حاصل از اجرای هر پنج معادله را خلاصه می‌کند. شکل ۱۳ نیز مقادیر rMAE را برای دو مدل به صورت نموداری مقایسه می‌کند. با توجه به اختلاف زیاد بین مقادیر rMAE، محور عمودی نمودار در مقیاس لگاریتمی نمایش داده شده است.

جدول ۱۲: مقایسه‌ی جامع نتایج روی پنج معادله

برنده	کاهش rMAE	PINNsFormer		PINN		معادله
		rRMSE	rMAE	rRMSE	rMAE	
PINNsFormer	54%	0.0556	0.0120	0.0695	0.0262	برگرز
PINNsFormer	62%	0.6100	0.3790	0.9910	0.9916	آلن-کان
PINNsFormer	95%	0.0047	0.0032	0.0926	0.0601	شرودینگر
PINNsFormer	> 99%	0.0001	0.0001	0.5124	0.3012	کوراموتو-سیواشینسکی
PINNsFormer	78%	0.0181	0.0184	0.0857	0.0820	KdV



شکل ۱۳: مقایسه‌ی rMAE مدل PINN و PINNsFormer روی پنج معادله (مقیاس لگاریتمی). در هر پنج معادله، میله‌ی نارنجی (PINNsFormer) کوتاه‌تر از میله‌ی آبی (PINN) است؛ برای کوراموتو-سیواشینسکی این تفاوت به بیش از سه مرتبه‌ی بزرگی می‌رسد.

۱.۵ الگوهای کلی مشاهده‌شده

اگر پنج معادله را کنار هم بگذاریم، عدد بهبود rMAE از 54% (برگرز) تا بیش از 99% (کوراموتو-سیواشینسکی) در نوسان است، بازه‌ای نسبتاً وسیع که خودش نکته‌ی جالبی است. در شرودینگر، جایی که PINN استاندارد از ابتدا عملکرد نسبتاً قابل‌قبولی داشت ($rMAE \approx 0.06$)، PINNsFormer خطا را به 0.0032 رساند. در آلن-کان، جایی که PINN استاندارد عملاً شکست خورده بود ($rMAE \approx 0.99$)، PINNsFormer این عدد را به 0.38 رساند، بهبود بزرگ نسبی، اما همچنان خطای نسبتاً بالا در مقایسه با سایر معادلات. این دو مورد کنار هم نشان می‌دهند که مزیت معماری مبتنی بر توجه در همه‌جا یک شکل ندارد: جایی که PINN کاملاً شکست می‌خورد (آلن-کان)،

PINNsFormer می‌تواند وضعیت را بهبود دهد بدون آنکه لزوماً به دقت بالا برسد؛ جایی که PINN از ابتدا نسبتاً موفق است (شرودینگر)، بهبود بیشتر در سطح جزئیات دقت رخ می‌دهد. معادله‌ی کوراموتو-سیواشینسکی از این دو الگو فاصله دارد: خطای PINN استاندارد در حد متوسط بود (0.30)، اما PINNsFormer آن را تا 0.0001 پایین آورد، بیشترین بهبود کل پروژه. بخش ۱.۴.۴ این مورد را با توجه به تطبیق مقیاس فیزیکی دامنه با ابرپارامترهای ثابت مولد دنباله بررسی کرده است.

۲.۵ تحلیل هزینه‌ی محاسباتی

جدول ۱۳ زمان آموزش هر مدل را نشان می‌دهد.

جدول ۱۳: مقایسه‌ی زمان آموزش			
معادله	زمان PINN	زمان PINNsFormer	نسبت
برگرز	4 دقیقه 1 ثانیه	28 دقیقه 25 ثانیه	$\sim 7.1 \times$
آلن-کان	1 دقیقه 28 ثانیه	32 دقیقه 53 ثانیه	$\sim 22.4 \times$
شرودینگر	7 دقیقه 12 ثانیه	49 دقیقه 59 ثانیه	$\sim 6.9 \times$
کوراموتو-سیواشینسکی	2 دقیقه 53 ثانیه	53 دقیقه 57 ثانیه	$\sim 18.7 \times$
KdV	10 دقیقه 38 ثانیه	1 ساعت 49 دقیقه	$\sim 10.3 \times$
میانگین			$\sim 13.1 \times$

۶ جمع‌بندی

در این پروژه، مدل PINNsFormer برای پنج معادله، برگرز، آلن-کان، شرودینگر غیرخطی، کوراموتو-سیواشینسکی و کورتوگ-دووریس، پیاده‌سازی و با PINN استاندارد مقایسه شد. چهار مشاهده‌ی اصلی این مقایسه را می‌توان این‌طور خلاصه کرد: اول، در هر پنج آزمایش rMAE مدل PINNsFormer از مدل مبنا پایین‌تر بود. دوم، اندازه‌ی این کاهش یکنواخت نبود و از 54% (برگرز) تا بیش از 99% (کوراموتو-سیواشینسکی) نوسان داشت. سوم، معادله‌ی کوراموتو-سیواشینسکی، تنها معادله با مشتق مرتبه‌ی چهارم در این مجموعه، بیشترین بهبود را نشان داد، در حالی که در آلن-کان با وجود بهبود قابل‌توجه، خطای نهایی همچنان بالا ماند. چهارم، این بهبود دقت رایگان نبود: زمان آموزش PINNsFormer به‌طور میانگین حدود 13 برابر PINN استاندارد بود. بررسی اجزای تابع هزینه هم نشان داد که کاهش خطای نهایی همیشه از کاهش باقی‌مانده‌ی معادله نمی‌آید؛ در شرودینگر، بخش زیادی از بهبود به شرط اولیه مربوط بود.

با این حال، چند محدودیت روش‌شناختی باید در تفسیر این نتایج در نظر گرفته شود: تمام آزمایش‌ها با یک بذر تصادفی اجرا شدند، تعداد و تنوع ابعادی معادلات بررسی‌شده محدود است، محدودیت حافظه‌ی GPU تراکم نقاط کولوکیشن معادله‌ی کوراموتو-سیواشینسکی را کاهش داد، و تنها یک مجموعه ابرپارامتر برای هر معادله آزموده شد. با در نظر گرفتن این محدودیت‌ها، آنچه از این پروژه برمی‌آید این است: ساختار دنباله‌ای همراه با مکانیزم توجه، دست‌کم در این پنج مورد، توانست دقت PINN استاندارد را در ازای هزینه‌ی محاسباتی قابل‌توجهی بهبود دهد، اما این بهبود در همه‌ی مسائل به یک اندازه یا با یک مکانیزم رخ نداد.

منابع

- [1] Zhao, Z., Ding, X., & Prakash, B. A. (2024). PINNsFormer: A Transformer-Based Framework for Physics-Informed Neural Networks. *International Conference on Learning Representations (ICLR)*.
- [2] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.
- [3] Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5), 987–1000.
- [4] Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3), 218–229.
- [5] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2020). Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*.
- [6] Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1), 208–228.
- [7] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., & Courville, A. (2019). On the spectral bias of neural networks. *International Conference on Machine Learning (ICML)*, 5301–5310.
- [8] Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., & Mahoney, M. W. (2021). Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, 34.
- [9] Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5), A3055–A3081.
- [10] Wang, S., Yu, X., & Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449, 110768.
- [11] McClenny, L., & Braga-Neto, U. (2020). Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*.
- [12] Wang, S., Sankaran, S., & Perdikaris, P. (2022). Respecting causality is all you need for training physics-informed neural networks. *arXiv preprint arXiv:2203.07404*.

- [13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [14] Dağ, İ., Irk, D., & Saka, B. (2005). A numerical solution of the Burgers' equation using cubic B-splines. *Applied Mathematics and Computation*, 163(1), 199–211.
- [15] Khalid, N., Abbas, M., Iqbal, M. K., et al. (2020). A numerical investigation of Caputo time fractional Allen–Cahn equation using redefined cubic B-spline functions. *Advances in Difference Equations*, 2020, 158.
- [16] Gardner, L. R. T., Gardner, G. A., Zaki, S. I., & El Sahrawi, Z. (1993). B-spline finite element studies of the non-linear Schrödinger equation. *Computer Methods in Applied Mechanics and Engineering*, 108(3–4), 303–318.
- [17] Lakestani, M., & Dehghan, M. (2012). Numerical solutions of the generalized Kuramoto–Sivashinsky equation using B-spline functions. *Applied Mathematical Modelling*, 36(2), 605–617.
- [18] Jain, P. C., Shankar, R., & Bhardwaj, D. (1997). Numerical solution of the Korteweg-de Vries (KdV) equation. *Chaos, Solitons & Fractals*, 8(6), 943–951.